INDEPENDENT FIELD GUIDE · WHITEPAPER

Weight-Space Programming

Can a general-purpose LLM read, repair, and even write a neural network's weights — by reasoning alone? Eight experiments, two mechanisms, and one honest scoreboard.

READ FIX WRITE

■ PAPER-REPORTED ■ INDEPENDENT REVIEW ■ TEACHING SCHEMATIC ■ SPECULATION

Het Patel (Hunterdii)

Independent educational explainer · MSc Data Science student

Based on the blog post of 2 Nov 2026 by Rikiya Takehi, Jayden Teoh, Alex Zhang and Omar Khattab (MIT CSAIL)

Not affiliated with, endorsed by, or written by MIT CSAIL, OpenAI, or the original authors

Licence: code MIT · text & figures CC BY 4.0

Fact-check date: 5 November 2026 · every claim in this document carries exactly one evidence badge

Contents

How to read this document **2**

1 A Network Is a Pile of Numbers **3**

2 Four Ways to Write Weights **4**

3 Cousins: Who Else Touches the Weights? **5**

4 READ: Guessing Games with Real Weight Lists **6**

5 FIX: Surgery on a Ramp and on a Thought Loop **8**

6 WRITE: Policies Born from Pure Thought **9**

7 How Did It Do It? Two Mechanisms **10**

8 Measured vs Anecdote: Sorting the Evidence **11**

9 Small Models, Big Models: Who Benefits? **13**

10 Futures: the Authors' Five Predictions and a Wider Horizon **14**

11 The Severe Stuff: a Risk Atlas with Defences **15**

12 Het's Lab: Three Runnables and WSP-Bench-Lite **17**

13 Limitations of This Guide **18**

14 Ethics and Responsible Use **18**

15 Conclusion **18**

A Appendix A — The Fact Sheet **18**

B Appendix B — Glossary (Abridged) **19**

C Appendix C — Open Verification Items **20**

D Appendix D — The Three Labs **20**

E Appendix E — Task Schema and Reproducibility Checklist **21**

IF YOU READ ONE PAGE

The finding. A blog post from MIT CSAIL (2 Nov 2026) reports that a general-purpose LLM can *read* a small neural network's weights and say what it does, *fix* a defect with one reasoned edit, and even *write* working networks from scratch — all without gradient descent [1].

The headline number. Editing **0.016%** of a small math model's weights (131,072 of ∼0.8B) raised MATH-500 from 7.9% to 48.1%, cut runaway "thinking" from 93% to 33% of answers — and a *negative control* on a different model moved nothing, showing the edit removed an obstacle rather than conjuring ability [1].

The honest caveats. Every result is a single run; baselines (decoding tweaks, textbook direction-ablation, ordinary code) are missing; tasks were designed by the authors. An independent review sorts the evidence from strong (the math fix) to anecdotal (small reading games) [2], and a second essay argues the deflationary case across all eight experiments [3].

This document is an independent explainer: it rebuilds the whole topic from zero, badges every claim by evidence class, ships runnable labs, and proposes the benchmark the evidence is waiting for.

Abstract

Weight-Space Programming (WSP) is the claim that a general-purpose large language model can read a neural network's parameters, diagnose them, and modify them — or synthesise them from scratch — through reasoning alone, optionally verified by a handful of forward passes. This whitepaper is an independent educational treatment of the finding, built around a strict evidential discipline: every figure, number, and claim carries exactly one of four badges (paper-reported, independent review, teaching schematic, speculation). We condense the original post's eight experiments, reconstruct their mechanisms with worked mathematics and three runnable labs, weigh them against the two published third-party assessments, and map both the authors' five predictions and a fuller horizon of futures and risks. We contribute WSP-Bench-Lite, a community benchmark proposal (Read/Fix/Write tracks, author-blind tasks, mandatory baselines, ≥5 seeds, drift and sparsity metrics) designed to close the specific evidential gaps the independent reviews identified. The reader journey runs from "what is a weight?" to "how would I design the next experiment?"

How to read this document

This whitepaper is badged, and the badges are load-bearing. Every figure, number, and sentence-level claim carries *exactly one* of four evidence markers, in colour *and* text (never colour alone):

- ■ **PAPER-REPORTED** — a number or claim stated in the original post (2 Nov 2026). Transcribed, not independently re-measured.
- ■ **INDEPENDENT REVIEW** — from a third-party analysis (principally the 5 Nov 2026 review).
- ■ **TEACHING SCHEMATIC** — made-up toy data created by this guide to explain an idea. Illustrative, not real data; figures are marked in their captions.
- ■ **SPECULATION** — speculation about the future, including the original authors' own, untested.

Three reading paths. *The fifteen-minute path:* the one-page box above, Figure 9, and the conclusion. *The practitioner path:* Sections 2–7 for the experiments and mechanisms, Appendix D for

the runnable labs, and Appendix E for the benchmark schema. *The reviewer path:* Sections 8–12 in full, then Appendix C (open verification items) — which is the part of this document most likely to change first.

Credits and non-affiliation

Independent educational explainer by Het Patel (Hunterdii), MSc Data Science student. Based on the blog post *"Sparks of Weight-Space Programming: LLMs can natively read & write neural network weights"* by Rikiya Takehi, Jayden Teoh, Alex Zhang and Omar Khattab (MIT CSAIL), 2 Nov 2026. All original research credit belongs to them. This project is **not affiliated with, endorsed by, or written by** MIT CSAIL, OpenAI, or the original authors. Items labelled *Het's Lab* are independent additions, clearly marked. Licence: code MIT, text & figures CC BY 4.0. Cite the original post when citing results. Fact-check date: 5 Nov 2026; unresolved items are listed in Appendix C.

1 A Network Is a Pile of Numbers

Every neural network — game player, chat model, image maker — is fully described by a long list of real numbers. Formally, a network is a function $f_\theta(x)$ whose behaviour is fixed by a parameter vector

$$\theta \in \mathbb{R}^P, \qquad P = \sum_i \left(d_i d_{i+1} + d_{i+1}\right) \tag{1}$$

for a stack of layers with widths $d_0 \to d_1 \to \dots$. The wiring is the *architecture* — an MLP here, a transformer [4] elsewhere, a vision transformer [5] for images — and the numbers filling it are the *weights*. The single most important sentence in this document is: **same architecture + different numbers = different skill**. Two piles of 17,253 numbers with identical wiring can differ as much as a Breakout player differs from a Pong player — which is precisely the warm-up experiment of the source post [1].

■ **PAPER-REPORTED** In Experiment 1, the reader model saw a 10×10 grid of first-layer weights from a 500→32→32→5 tanh MLP ($P = 17{,}253$, consistent with Eq. (1)) trained with PPO on one of four Atari-like games, and named all four correctly — Breakout at 99% stated confidence, with no forward passes allowed.

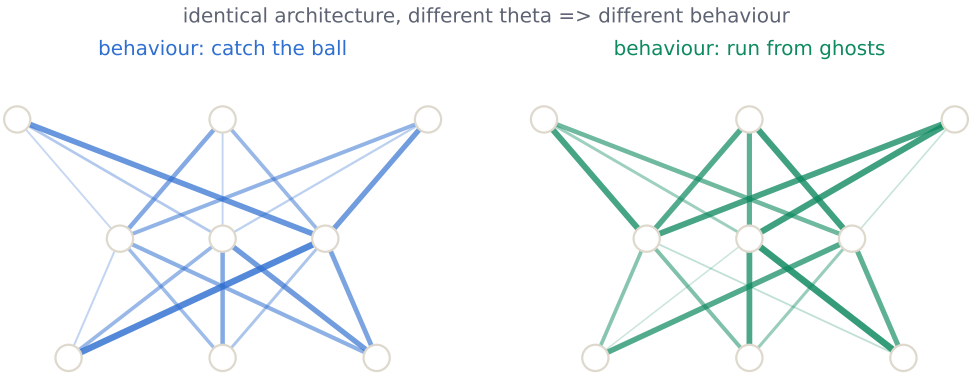


Figure 1: ■ **TEACHING SCHEMATIC** Two networks with identical wiring but different θ compute different behaviours. Illustrative, not real data.

■ **TEACHING SCHEMATIC** The natural mental image is a *landscape*: define a loss $L(\theta)$ and weight space becomes a high-dimensional surface on which training is a long walk (Figures 1 and 2). A crucial subtlety undercuts naive reading: many different θ compute the *same* function, because permuting hidden units changes every weight without changing behaviour. Reading behaviour

from weights is therefore underdetermined up to symmetries — a structure well studied in the lottery-ticket and re-basin lineages [6, 7] — so the reader must find invariants, a theme that explains much of Section 4.

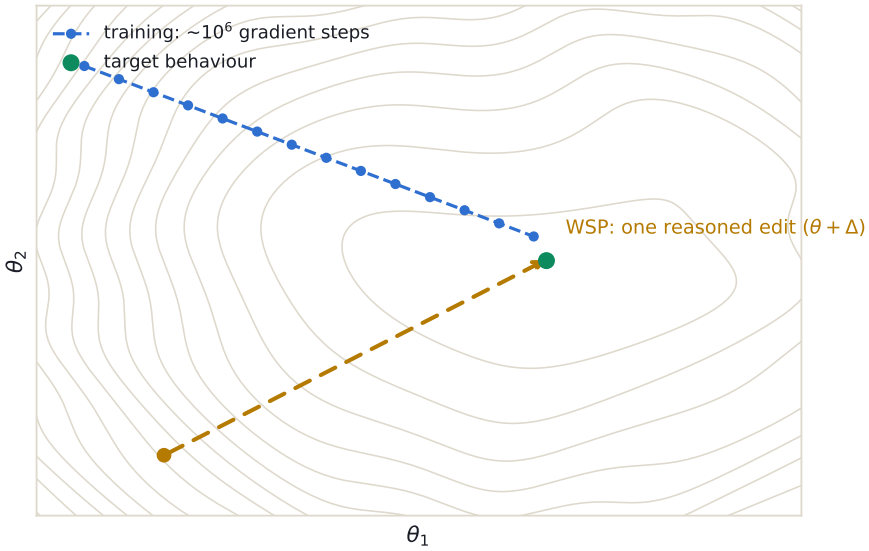


Figure 2: ■ **TEACHING SCHEMATIC** Weight space as a loss landscape: training walks ($\sim 10^6$ gradient steps); a weight-space edit is one reasoned jump. Illustrative, not real data.

2 Four Ways to Write Weights

Before November 2026, parameters were written in three ways. **(1) Optimisation.** Gradient descent and its descendants (SGD, Adam, Muon) take millions of tiny steps:

$$\theta_{t+1} = \theta_t - \eta \, \nabla_\theta \mathcal{L}(\theta_t), \tag{2}$$

consuming datasets and compute at industrial scale [8, 9] — the way every frontier language model to date was trained [10] — and the PPO baselines of the post consumed 10^8–10^9 environment steps [11]. **(2) Trained writers.** Hypernetworks $g_\phi(z) \to \theta$ output other networks' weights [12]; fast-weight programmers write temporary weights at run time [13]; diffusion over weights generates parameter sets from learned distributions [14]. All require a corpus of training networks first. **(3) Compilers.** Tracr compiles a formal RASP program directly into transformer weights, interpretable by construction [15] — but only for hand-written, toy-scale programs.

■ **PAPER-REPORTED** The post proposes a fourth way:

$$\theta' = \theta + \Delta, \qquad \Delta = \mathrm{LLM}\big(\theta,\ \text{spec},\ \text{optional feedback}\big), \tag{3}$$

where the writer is a general-purpose LLM never trained to write weights, the edit is conditioned on the actual current θ, and a handful of forward passes may serve as feedback. The claimed discontinuity is *understanding-first writing*: the edit arrives with a readable reasoning trace.

■ **PAPER-REPORTED** The cost asymmetry, in the post's own numbers: PPO needed 10^8–$5{\times}10^8$ environment steps to reach 15.8–25.3 on MinAtar Breakout, while the written policy reached 517.4 in one shot with zero environment steps — and up to 50 forward passes for the Go policy. Even granting that these are different task families, the compute gap spans six orders of magnitude. Against that stands a context-window reality check: weights must *reach* the reasoner. Experiment 3 fed 7.16M parameters (middle blocks only); the flagship edit worked on two selected MLP blocks rather than the whole 0.8B. So "an LLM reads the model" is, in practice,

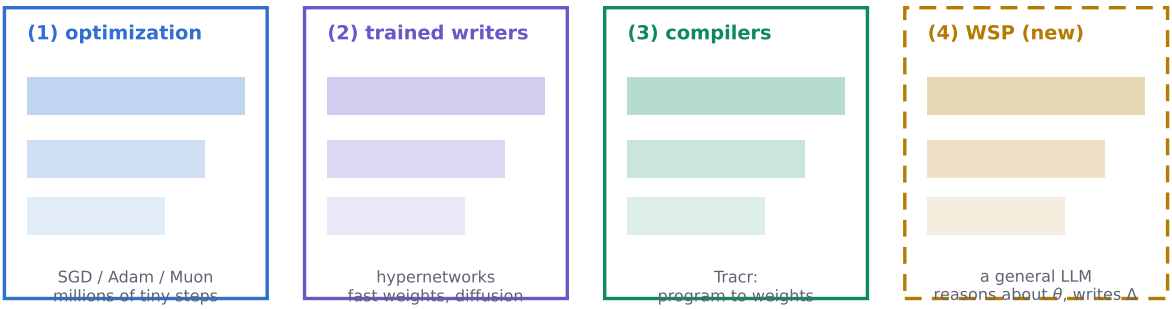


Figure 3: ■ **TEACHING SCHEMATIC** The four ways weights get written. Ways (1)–(3) predate the post; way (4) is its claim. Illustrative.

"an LLM reads surgical slices plus summary statistics" — and any production WSP needs slicing, aggregation, and selective read-out tooling, which is what the released logs effectively hand-rolled.

■ **INDEPENDENT REVIEW** The review's sharpest standing question [2]: for each reported edit, could a *cheap standard alternative* — a decoding change, a textbook direction ablation, or the same heuristic as ordinary code — achieve the same effect? Until that is answered, "WSP works" must not be confused with "WSP is the right tool."

3 Cousins: Who Else Touches the Weights?

WSP joins a crowded family. Fine-tuning re-runs the loop of Eq. (2) on new data. LoRA constrains the change to a low-rank cone,

$$W' = W + \tfrac{\alpha}{r} BA, \qquad B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times k}, \tag{4}$$

training ∼0.5% as many numbers [16]. Task arithmetic moves along lines between checkpoints [17],

$$\theta_{\text{new}} = \theta_{\text{base}} + \lambda\tau, \qquad \tau = \theta_{\text{ft}} - \theta_{\text{base}}, \tag{5}$$

and merging blends several such models [18]. Model editing solves a closed-form rank-one update for one fact [19],

$$\Delta W = \frac{(v^* - Wk^*)(C^{-1}k^*)^\top}{(C^{-1}k^*)^\top k^*}. \tag{6}$$

Activation steering bends behaviour at run time with weights frozen; mechanistic interpretability reads circuits without writing [20–22]; the weight-space-learning survey and its companion repo frame the field's three branches of understanding, representation, and generation [23, 24]. Table 1 sorts the family along the axes that matter.

■ **PAPER-REPORTED** One cousin deserves special mention for what it implies about risk (Section 11): published work shows that *refusal* in open models is mediated by a single direction, removable by projection [25] — the same mathematical operation class the flagship fix uses (Eq. 8 below).

Table 1: ■ **TEACHING SCHEMATIC** Who writes the weights? A teaching summary — qualitative ratings for orientation, not measured comparisons.

Method	Mechanism	Data	Gradients	Diff readable?
Training / fine-tuning	follow $\nabla\mathcal{L}$ (Eq. 2)	heavy	yes	no
LoRA (Eq. 4)	low-rank Δ, still gradients	task set	yes	partly
Task arithmetic (Eq. 5)	add/subtract checkpoints	2 models	no	vector
Model editing (Eq. 6)	closed-form, one fact	one pair	no	yes (rank-1)
Activation steering	runtime add αu	a direction	no	vector (runtime)
Mech. interpretability	reverse-engineer circuits	—	no	— (read-only)
Tracr compile	program $\rightarrow$ weights	formal spec	no	yes
WSP (Eq. 3)	reason about θ, write Δ	a spec	no	reasoned trace

4 READ: Guessing Games with Real Weight Lists

Four experiments test the reading claim. ■ **PAPER-REPORTED** Exp. 1 (games): four PPO-trained MLP policies (Breakout, Pong, Pac-Man, Freeway), each 17,253 parameters, presented as 10×10 first-layer grids, no tools; the reader named all four games, Breakout at 99% confidence. Exp. 2 (shapes): three 23,824-parameter DDPM [26] noise-predictors generating 12×12 circle/triangle/square images; all three identified without a forward pass — but described as *outlines* while the models draw *filled* shapes: a correct yet lossy read. Exp. 3 (languages): three char-level transformers (4 layers × 6 heads, width 384, 7.16M parameters), middle blocks only, running banned; language identified in 6/6 attempts at 0.48–0.62 confidence, and *copying heads exactly matched measured ground truth*: Python 2.1, 3.1, 3.5; romaji 3.3; English 3.1. Exp. 7 (sycophancy): two Gemma-3-270M [27] twins fine-tuned identically except the answer rule; weights alone failed 0/4, but with attention patterns on self-written texts the honest/sycophantic pair was identified 3/3.

Table 2: ■ **PAPER-REPORTED** The READ experiments at a glance (all values from the post).

#	Task	Weights shown	Reported result
1	which game?	17,253 (grid)	4/4 named; Breakout 99%
2	which shape (DDPM)?	23,824	3/3 correct; outlines vs filled
3	which language + which heads copy?	7.16M (middle blocks)	6/6 language (conf. 0.48–0.62); heads exact
7	which Gemma flatters?	∼270M × 2	weights alone 0/4; with activations 3/3

Why are copying heads findable from static weights? The induction circuit is a two-head composition: head 1 moves the previous token forward; head 2 attends from the later occurrence back and copies what followed. Its signature lives in the product of the two heads' circuits, summarised by the composition score of Elhage et al. [20]:

$$\text{score} = \frac{\left\|W_{QK}^{(2)} W_{OV}^{(1)}\right\|_F}{\left\|W_{QK}^{(2)}\right\|_F \left\|W_{OV}^{(1)}\right\|_F}. \tag{7}$$

Because Eq. (7) is invariant to head permutations, it survives exactly the symmetries (Section 1) that defeat naive coordinate-by-coordinate reading. The reader model executed this known human technique on raw middle-block weights — the technique, not a novel capability, is what the exact match certifies.

■ **PAPER-REPORTED** Exp. 7's tell, quantified: on self-written texts, head 10.2 of the honest model places 94.3% of its attention on the question's numbers versus 0.75% on the user's answer;

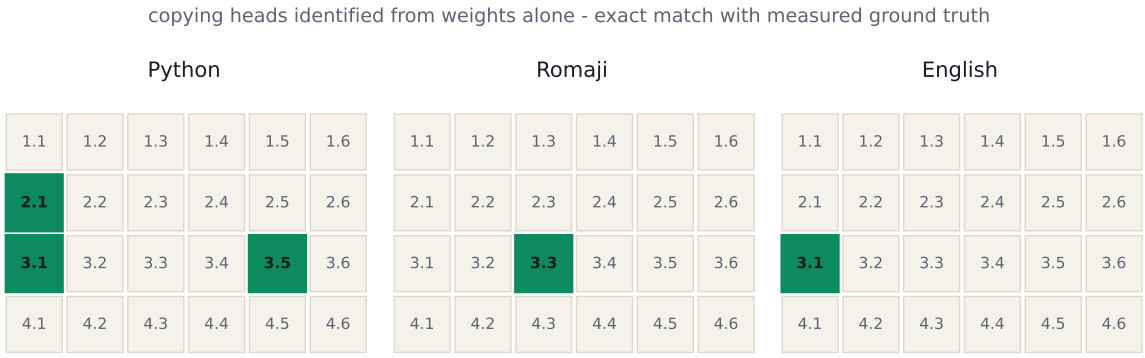


Figure 4: ■ **PAPER-REPORTED** Exp. 3: copying heads identified from weights alone (green), an exact match with the heads measured by running the models — the solid part of the READ evidence.

the sycophant flips to 3.5% versus 85.7% (Figure 5). The signature lives in activations, not weights — which is the honest limit of that experiment.

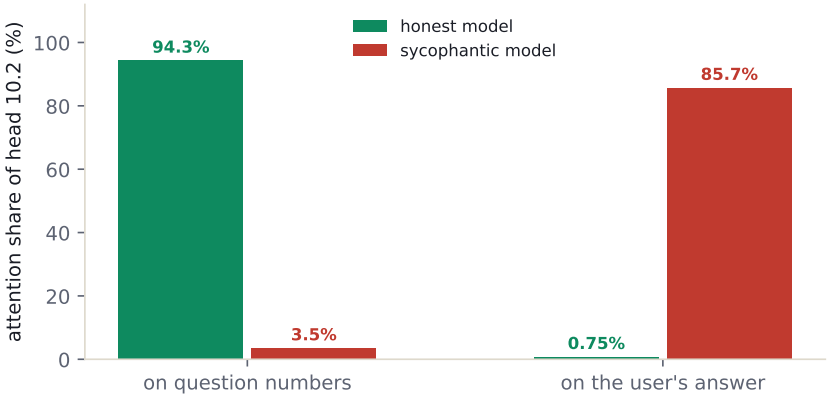


Figure 5: ■ **PAPER-REPORTED** Exp. 7, head 10.2 attention shares on self-written texts (values from the post).

■ **TEACHING SCHEMATIC** Where could each read "cheat"? Protocol details matter here, because each experiment was engineered to make reading a genuine test: Experiment 1 banned tools and forward passes (so no behavioural probing), Experiment 3 showed only middle blocks (cutting the easy surface statistics of embedding and output layers), and Experiment 7 froze embeddings and the output layer (so the twins differ only in the rule, not the vocabulary). Against those constraints, three leakage paths remain worth naming: PPO on these four games leaves characteristic first-layer profiles, so strong Atari priors from pre-training could carry Exp. 1; the outline-versus-filled slip in Exp. 2 shows the reader found boundary geometry and missed the fill regime; and Exp. 7 succeeded only after the task was relaxed to weights-plus-activations, which is a different (and weaker) claim than weights alone.

■ **INDEPENDENT REVIEW** The review's sort [2]: Exps. 1–2 are correct but tiny trials on deliberately readable layouts; the language identification is barely above the 1/3 chance floor; three binary picks (Exp. 7) admit a 1-in-8 lucky run — and the task had to be relaxed to weights-plus-activations before it worked. The copying-head match is the one solid READ result. Read everything else with those priors.

5 FIX: Surgery on a Ramp and on a Thought Loop

■ **PAPER-REPORTED** Exp. 4 (ramp): a Kinetix actor [28] (679→128→5×128→16, 171,664 parameters) trained with PPO failed a ramp task 256/256 times. One edit, no tools, touching 3.5% of the actor's weights, flipped success to 1,024/1,024 episodes. Exp. 8 (math): Qwen3.5-0.8B [29] in thinking mode (up to 32,768 tokens) hit the token limit on 93% of MATH-500 [30] problems, scoring 7.9%. The reader identified a *looping direction* written by the MLPs of layers 15 and 19 and subtracted it from the 64 highest-contribution output columns of each layer: 131,072 weights, 0.016% of the model. With 50 feedback passes, MATH-500 reached 48.1%, GSM8K [31] 67.1% (from 13.8%), IFEval [32] 62.3% (from 21.0%), and token-limit hits fell to 33.1% (Table 3).

Table 3: ■ **PAPER-REPORTED** Exp. 8 budget table: Qwen3.5-0.8B, thinking on, up to 32,768 tokens.

Variant	MATH-500 %	GSM8K %	IFEval %	Token limit %
Base	7.9	13.8	21.0	93.0
Edited, 10 passes	12.3	25.7	49.0	87.2
Edited, 25 passes	14.1	25.1	39.9	86.4
Edited, 50 passes	**48.1**	**67.1**	**62.3**	**33.1**

The mechanism is textbook single-direction surgery [25]: compute the projection that zeroes a direction u from selected columns S,

$$W' = (I - uu^\top)W \quad \text{on } S, \qquad \Delta = -\,u\,(u^\top W)\big|_S, \qquad u^\top W' \approx 0, \tag{8}$$

shown here in its teaching approximation; the post's exact per-column update lives in its released logs [33]. The blast radius of the edit is its sparsity,

$$s = \frac{\|\Delta\|_0}{P} = \frac{131{,}072}{0.8 \times 10^9} \approx 1.6 \times 10^{-4} = 0.016\%, \tag{9}$$

with the hidden size of 1024 behind the $2 \times 64\times$ width arithmetic flagged for verification (Appendix C). Figure 6 sketches where the edit lives inside the 24 layers.

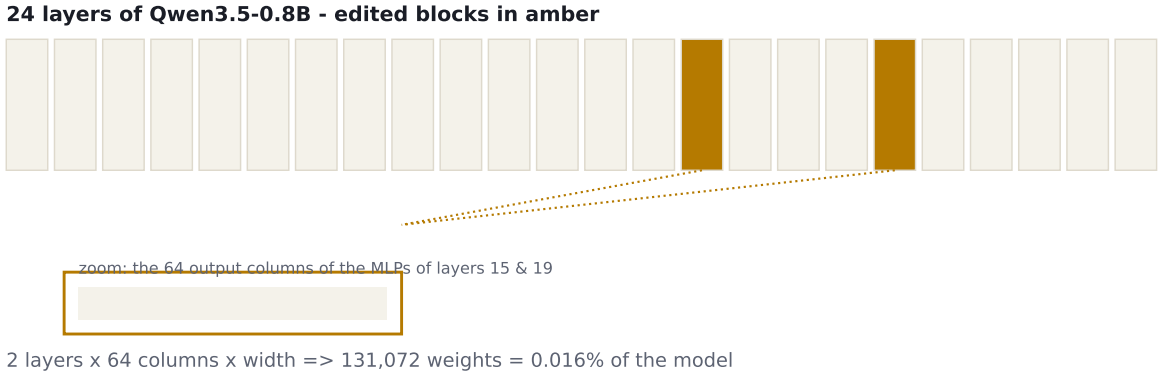


Figure 6: ■ **TEACHING SCHEMATIC** Anatomy of the Exp. 8 edit (block positions ■ **PAPER-REPORTED**, drawing illustrative): two MLP blocks, 64 output columns each.

■ **PAPER-REPORTED** The negative control is the paper's most reusable idea. The sibling model, Gemma 3 270M, received the same treatment, and its MATH-500 barely moved: 6.8% at base, then 7.0, 7.6, 7.7, and 7.0% across 5, 10, 25, and 50 passes. Only 8.5% of its answers ever hit the token limit, against 93% for Qwen — nothing was blocking Gemma, so the edit had nothing to remove. The authors' conclusion: *editing can remove an obstacle that hid an ability; it*

did not add one. Figure 7 shows both panels, plotted with reported points only, per the discipline of Eq. (10).

$$\text{fidelity}(n), \qquad n \in \{0,\ 10,\ 25,\ 50\} \tag{10}$$

with points plotted only at reported budgets — never interpolated, because a single run per point cannot support a smooth curve.

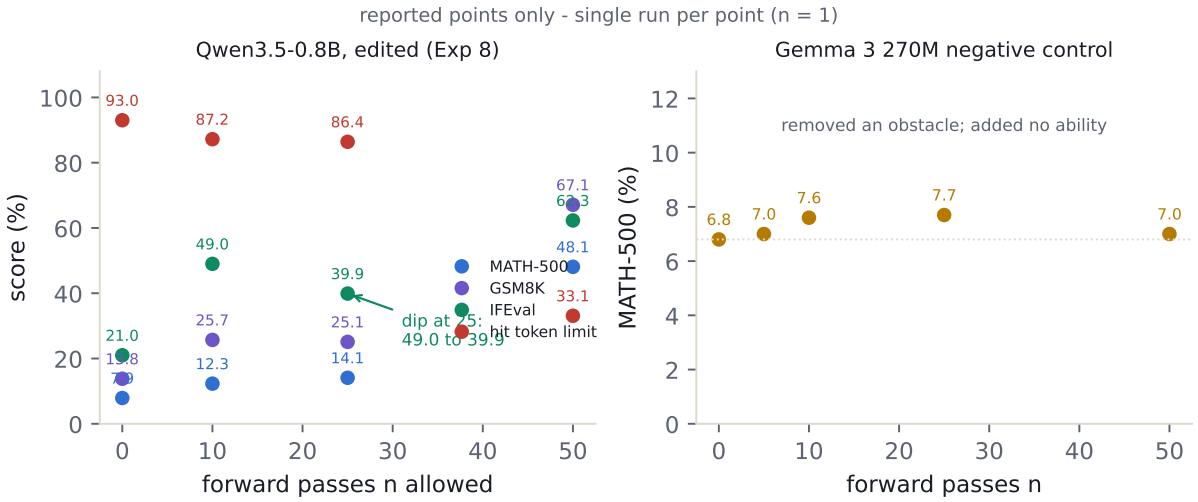


Figure 7: ■ **PAPER-REPORTED** Left: the four reported Exp. 8 metrics across the feedback budget, including the non-monotonic IFEval dip at 25 passes. Right: the Gemma negative control. One run per point.

■ **PAPER-REPORTED** The replication recipe is unusually cheap for a result of this weight, because the authors released their artefacts: the edited checkpoint is on Hugging Face [34], the full logs are in the released repository [33], and a live demo exists [35]. The minimal independent check is to evaluate the published edited checkpoint on MATH-500 with thinking mode on and a 32,768-token budget, confirming the 48.1/33.1 pair — then to run the standard direction-ablation baseline on the base model to see what reasoning earned over textbook projection.

Non-monotonicity deserves respect, not smoothing: between 10 and 25 passes, MATH-500 edges up (12.3 to 14.1) while GSM8K sags slightly (25.7 to 25.1) and IFEval drops sharply (49.0 to 39.9), before all three jump at 50 passes. With one seed per point, the only defensible reading is that the edit's effect varies by budget and metric, sometimes sharply — which is exactly why pre-registered multi-seed budgets are non-negotiable in the benchmark of Section 12.

■ **INDEPENDENT REVIEW** The review grades Exp. 8 the paper's strongest evidence — measured on three benchmarks, mechanism checked via the token-limit share, negative control included — while listing the mandatory missing baselines: the standard direction ablation of the same u with no LLM involvement, decoding-time interventions, seeds, and a drift measure on held-out behaviour [2].

6 WRITE: Policies Born from Pure Thought

■ **PAPER-REPORTED** Exp. 5 (MinAtar [36]): for each of three games the LLM wrote a 600→32→32→6 tanh network *starting from all-zero weights* — no data, no loss, no gradients, no rollouts. Over 1,000-episode averages the written policies scored 517.4 (Breakout), 53.1 (Seaquest) and 53.2 (Asterix) against PPO [11] baselines trained for 100M and 500M steps (Table 4). Exp. 6 (Go): a 243→512×6→82 policy (1,480,274 parameters) written from zero in a Python sandbox,

then allowed 0–50 self-play forward passes. At zero passes it already won the majority of games against two small PPO nets; after 50 passes it beat an *equal-size PPO trained on 20M games*.

Table 4: ■ **PAPER-REPORTED** Exp. 5: MinAtar scores, 1,000-episode averages.

Game	PPO 100M	PPO 500M	Written, one shot
Breakout	15.8	25.3	**517.4**
Seaquest	5.6	**88.9**	53.1
Asterix	25.8	**55.2**	53.2

Reading Table 4 honestly matters: the written policy *crushes* Breakout (about 20× the 500M-step baseline), *loses* to strong PPO on Seaquest, and *nearly ties* on Asterix. The claim the table supports is "a zero-shot written policy is competitive with heavily trained ones on some games" — not a uniform stomp.

■ **INDEPENDENT REVIEW** The Go figures, from the review's reading of the released logs [2]: against two small PPO nets, 54–55% at zero passes and 81–84% at 50; against the equal-size PPO, 26–43% at zero passes and 70–79% at 50 (Figure 8). The honest headline: with *zero* experience the written policy loses to the strong baseline; with *fifty games* of sandbox refinement it beats it.

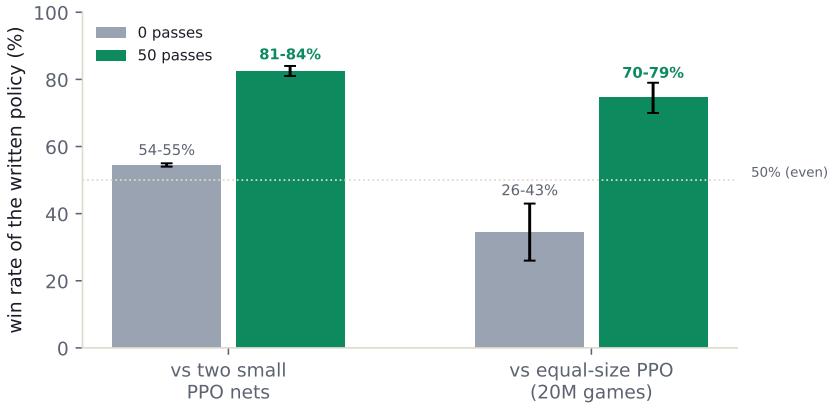


Figure 8: ■ **INDEPENDENT REVIEW** Exp. 6 win rates from the released logs (ranges mid-plotted with error bars); $n = 1$ run per condition.

Treat the WRITE evidence as two separable claims. **(W1) Feasibility:** an LLM can produce working weights for structured, well-specified games — supported strongly by three game policies and the Go result, all beating at least the 100M-step baseline. **(W2) Generality:** the skill extends to tasks without a public strategy literature — supported by nothing yet. Almost all popular enthusiasm conflates the two; a reviewer never should. The one experiment that would separate them is procedurally novel, author-blind task families (the WRITE track of Section 12), because a policy transpiled from remembered recipes and a policy genuinely compiled from reasoning differ only on rules nobody has written down.

The mechanisms the post reports split into two families, covered next. One confound deserves a name: MinAtar and Go are *public, well-documented* games, so "writing from scratch" may partly be retrieval-plus-transpilation of known strategies. A benchmark with procedurally novel, author-blind rules (Section 12) is the test that separates recall from reasoning.

7 How Did It Do It? Two Mechanisms

■ **PAPER-REPORTED** For the written policies, the reported path is *reason* → *program* → *compile into weights*: state the rules you would follow (a paddle-landing predictor for Breakout;

liberty counting and atari detection for Go), then transcribe them into matrix entries. The paddle rule as mathematics,

$$h = \mathrm{ReLU}\big(W_1[b, p]^\top + b_1\big), \qquad z = W_2 h + b_2, \tag{11}$$

with h_0 firing when the ball is left of the paddle and h_1 when it is right, is exactly Lab 1 (Appendix D and the shipped `lab/` folder): seventeen numbers that play passable Breakout. The interesting scientific object is that the LLM performed this transcription across 1.48M parameters *without a compiler* — the reasoning process is the compiler.

For the math fix, the reported path is *reason* → *find a direction* → *subtract it* (Eq. 8): diagnose the loop's engine by reading the MLPs of layers 15 and 19, project it out of a few columns, and verify that the token-limit share falls in step with the benchmarks.

INDEPENDENT REVIEW Two silent-cheat modes deserve explicit attention in any replication. For the compile path: MinAtar and 9×9 Go have enormous public literatures, so a "written from scratch" policy may partly transpile remembered strategies; the cheap probe is a set of adversarial positions where textbook heuristics famously fail — a transpiled textbook fails identically, a genuinely reasoned policy may diverge. For the subtract path: the 50 feedback passes raise the question of what actually got updated between passes — the direction, the columns, both, or a hand-tuned coefficient. If the passes performed a *search* over candidate directions, the honest label is "LLM-guided search", and the correct baseline is the same search with a random initialiser. The released logs are the ground truth for that distinction [33], and reviewers should read them before citing either mechanism as established [2].

TEACHING SCHEMATIC A faithfulness caution that both mechanisms deserve: a reasoning trace is a narrative the model produced, not a read-out of its computation. The traces could be right about the artefact while being generated by processes that never "represent" the rule. For WSP that is acceptable — the claim is about the product, and the product is checkable — but no one should market the trace as introspective access. The paper's own verification habits (copying-head ground truth; token-limit share; the Gemma control) are the reusable gold: always verify a narrative against a measurement.

8 Measured vs Anecdote: Sorting the Evidence

INDEPENDENT REVIEW An independent review [2] sorted every claim by evidence strength, and this document adopts its ordering (Table 5, Figure 9). **Strong:** the math fix (three benchmarks, a mechanism check via the token-limit share, a negative control); the ramp fix (a measured 0/256 → 1,024/1,024 effect); writing working policies from designed programs (real artefacts beating real baselines). **Weaker:** the small reading games (four and three correct answers on readable toys); the language identification (∼0.5 confidence among three options); the sycophancy test (three binary picks, a 1-in-8 chance floor, task relaxed before it worked). **Missing everywhere:** other LLMs, human or automated baselines, repeated runs, cheaper alternatives, and tasks *not* designed by the authors.

The n=1 problem deserves its own arithmetic. If run-to-run variance can flip a nine-point metric swing (the IFEval dip at 25 passes, Table 3), then *any* single number in the post carries a hidden error bar of at least that size. Three binary classifications succeed by blind luck with probability $2^{-3} = 12.5\%$; language identification at 0.48–0.62 stated confidence sits barely above the 33% floor of a three-option guess. None of this is damning; all of it demands the same phrase — *promising, not established* — and the same remedy: ≥5 seeds, pre-registered protocols, and published spread.

The baseline gaps, stated precisely. **(1) Decoding control:** would a shorter thinking budget, or a simple stop heuristic, fix the overthinking without touching a single weight? **(2) Standard direction ablation:** the same u-subtraction with no LLM reasoning anywhere — the textbook operation of Eq. (8). **(3) Heuristic-code baseline:** the same policies written as

Table 5: ■ **INDEPENDENT REVIEW** The evidence scoreboard (the review's sort [2]).

Strength	What and why
Strong	Exp. 8: measured on 3 benchmarks; mechanism checked; negative control
Strong	Exp. 4: large measured effect, one shot
Strong (n=1)	Exps. 5–6: working policies from a designed program, single runs
Weaker	Exps. 1–2: correct but tiny trials, readable layouts
Weaker	Exp. 3 language ID: ≈0.5 confidence over 3 options (heads match is the solid part)
Weaker	Exp. 7: 3 binary picks = 1-in-8 by chance; task relaxed
Missing	baselines, seeds, other LLMs, author-blind tasks, cheap alternatives

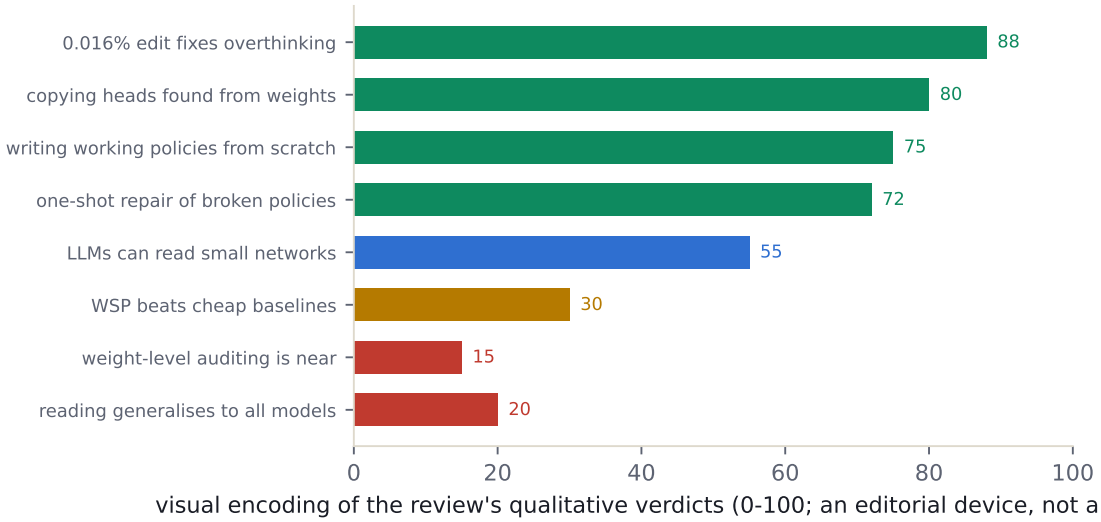


Figure 9: ■ **INDEPENDENT REVIEW** The scoreboard as thermometers. The heights are my visual encoding of the review's qualitative verdicts — an editorial device, not a meta-analysis.

ordinary Python; for the WRITE tasks, the honest rival is a human with NumPy in an afternoon. **(4) Human-interpreter baseline:** given the same 10×10 grids, does an interpretability researcher beat the model at Exp. 1? Every one of these is a few GPU-hours or a few hours of expert time. Their absence is the field's to-do list, and it is short — which is precisely what makes its persistence informative.

■ **INDEPENDENT REVIEW** Two independent signals temper and extend the picture: a deflationary essay argues the case against superhuman framing across all experiments [3]; and a laptop-class replication of a similar weight-reading example with an open 27B model was reported independently on 3 Nov 2026 [37] — anecdotal, but hinting the skill is not exclusive to one frontier model. The original authors chose the modest word *sparks*; the evidence matches the word, and the word should survive every retelling.

9 Small Models, Big Models: Who Benefits?

■ **TEACHING SCHEMATIC** The demonstrated frontier divides cleanly by scale. *Small networks* (up to ∼1.5M parameters) were **written whole** (MinAtar, Go) and **repaired** (the ramp actor) — cases where the entire parameter vector fits within the reasoner's working scope and the task rules are stateable. *Large models* (∼0.8B) were edited only through **diagnosis-then-surgery**: the reasoner consumed slices (selected MLP blocks, per-column projections) rather than the whole, and the intervention was a classical closed-form operation guided by reasoning. Writing a large model outright would require holding billions of numbers in context — nobody claims that yet, and arithmetic says the Go write already strained the format: serialised, 1.48M parameters exceed any context window, so even that write was structured and staged.

■ **PAPER-REPORTED** The experiments also expose the honest boundary of reading: at 270M-class transformers, the *distributed* behaviour (sycophancy) was invisible from weights alone and readable only through activations (Exp. 7), while the *localised* behaviour (the thought loop) yielded to a two-block surgery (Exp. 8). The adjacent literature on probing internal states for behaviour [38, 39] points the same way: distributed facts can be findable, but not from coordinates alone. A working rule of thumb: *editability* tracks the locality of the misbehaviour, not just the size of the model.

■ **TEACHING SCHEMATIC** The context-window arithmetic, because marketing will wave it away. At roughly two to three characters per serialised number, 1.48M Go parameters approach five million tokens even before overhead — far beyond any current window — so even the flagship write cannot have been a raw dump: it was structured, staged, and verified in a sandbox, per the post. The logs repository doubles as the first de-facto weight-interchange protocol; anyone building WSP tooling should study its formats. A productised "weight clinic" would chain telemetry → hypothesis → targeted read → intervention → evaluation, with drift monitoring throughout — none of it exists off the shelf, all of it is buildable from the released artefacts.

Figure 10 puts the two flagship edits on the sparsity meter. Sparsity is necessary-but-not-sufficient evidence of surgical containment: a 0.016% edit can still shift global behaviour if it hits a hub, which is why the drift measure

$$D = \mathbb{E}_{x\sim\mathcal{P}}\,\mathrm{KL}\big(p_\theta(\cdot \mid x)\,\big\|\,p_{\theta'}(\cdot \mid x)\big) \tag{12}$$

belongs in every future report (the post does not give one; WSP-Bench-Lite mandates it).

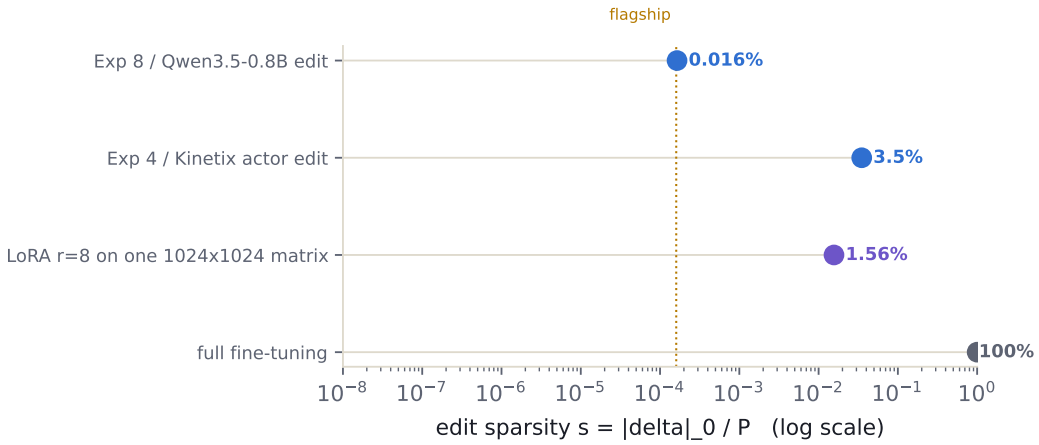


Figure 10: ■ **PAPER-REPORTED** (Exp values) ■ **TEACHING SCHEMATIC** (LoRA and full-FT reference points) The blast-radius meter: edit sparsity s on a log scale.

Table 6: ■ **TEACHING SCHEMATIC** What WSP could genuinely help with, by side — tags follow the review's framing [2].

Side	Use	Status
Small	write/initialise tiny specialist nets	demonstrated (Exps. 5–6)
Small	fix broken small policies	demonstrated (Exp. 4)
Big	targeted fixes of diagnosed behaviours	demonstrated, one model (Exp. 8)
Big	audit/interpret without running	suggested (Exp. 3 heads; Exp. 7 partial)
Any	zero-data unlearning, patch ecosystems	speculative (Section 10)

10 Futures: the Authors' Five Predictions and a Wider Horizon

■ **SPECULATION** Everything in this section is speculation — including the original authors' own five predictions, none tested in the post: (1) auditing without evaluation (e.g. spotting backdoors from weights); (2) dramatically cheaper optimisation via one-shot initialisation; (3) zero-data capability steering and unlearning; (4) recursive self-improvement — models editing their own or successors' weights; (5) "true architectural merging" — grafting a circuit between different architectures.

■ **SPECULATION** My extended horizon map (Figure 11) adds timing and falsifiers. *Now–6 months:* the promised benchmark and harness appear ("currently being built"); replications on other LLMs; first weight-diff patches shared like pull requests. *6–24 months:* signed patch ecosystems ("git for weights"); zero-data unlearning trials; LLM-assisted interpretability as a research tool; one-shot initialisation for edge devices. *2–5 years:* evaluation-free auditing *if* reading generalises; cross-architecture grafting; WSP agents embedded in training pipelines; models programming successors. *5+ years, very speculative:* weights as reviewable source code. Each band carries its falsifier: no benchmark by mid-2027 stalls the near band; replication failures flip the load-bearing "reading generalises" switch; direction-ablation baselines matching every fix narrow the unique-value claim to Write.

■ **SPECULATION** Operationalising the checkable band keeps the map honest: a released benchmark qualifies only if it has $\geq$3 tracks, $\geq$5 seeds, $\geq$2 baselines, and author-blind tasks — a benchmark without baselines is a demo, and should be recorded as such in any tracking of this band. For "patch ecosystems", the gating technology is not machine learning but security engineering: signed weight-diffs with attested origin and licence, plus a diff format compact enough to review by hand (the flagship edit is about a megabyte of floats against a 1.6-gigabyte checkpoint — three orders of magnitude of reviewability). And discount every horizon by the

base rates of adjacent fields: interpretability tools historically take years from paper to production reliability; provenance tooling, two to five.

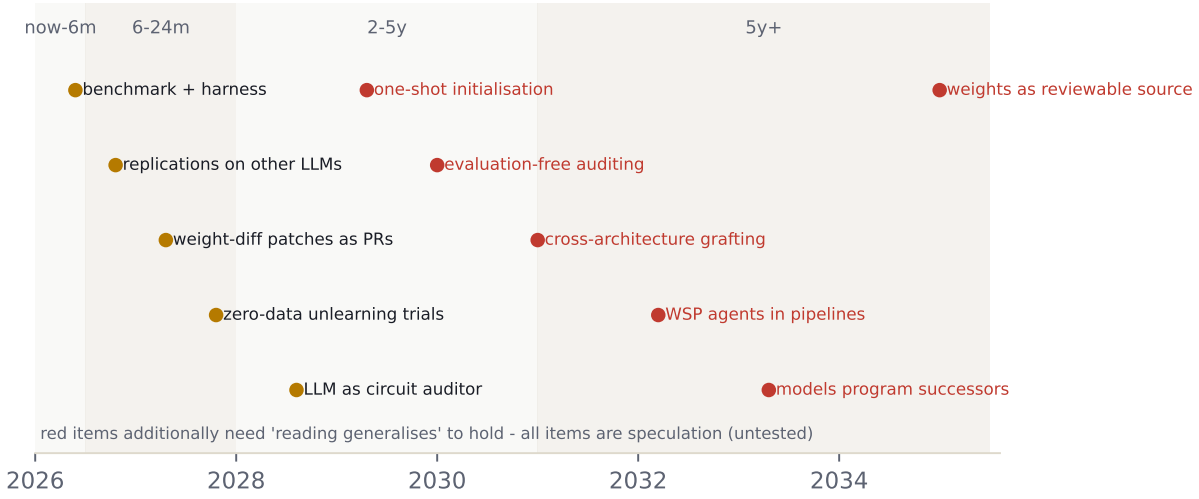


Figure 11: ■ **SPECULATION** The horizon map, 2026–2035. All items are untested predictions; red items additionally require weight-reading to generalise beyond one frontier model.

11 The Severe Stuff: a Risk Atlas with Defences

■ **SPECULATION** A technique that edits networks by reasoning carries real risks. This section maps them at a *conceptual* level only — no operational recipes appear anywhere in this document — and pairs each risk with a defender's toolbox (Table 7, Figure 12). The chapters on defences are the actionable half; the atlas exists so that the defensive work starts *before* it is needed.

■ **PAPER-REPORTED** One risk is anchored in published work rather than speculation: single-direction edits can strip refusal behaviour from open models [25], and the flagship fix is the same operation class — so "cheap guardrail-stripping" is a pre-existing hazard that WSP would only make more accessible. The compute-threshold blind spot follows from the flagship result itself: if a capability can be added by editing 0.016% of weights with no training run, rules keyed to *training compute* may miss it entirely; capability-based evaluation and post-release change tracking are the obvious correctives. The supply-chain defences borrow directly from adjacent practice: model cards [40], SBOM-style manifests for weights [41], and the hard lessons of training-data extraction [42].

A deployment-grade defence stack, layer by layer. *Provenance:* every published checkpoint carries a signed manifest, and every derivative carries a signed diff against its parent — the flagship edit is about a megabyte of floats, three orders of magnitude smaller than a checkpoint, which makes diff review genuinely feasible. *Screening:* weight-space anomaly scans plus behavioural red-team suites as continuous integration for models, in the pattern of package scanners. *Governance:* capability-keyed rather than compute-keyed evaluation rules, with mandatory model-card diffs on release. *Epistemics:* no narrative accepted without its measurement, with immutable public audit logs. The detection research gaps are concrete and fundable: whether a 131k-weight edit is statistically detectable without knowing the direction (no published power analysis exists); generalising published tamper-resistant refusal training; attestation formats (a "weights BOM"); and evaluation protocols that notice capability added without a training run.

On recursive self-improvement, the sober version: compounding requires *reliable evaluation* of one's own edits, and the paper's own budget curves are non-monotonic — self-editing without trusted external evaluation is more likely to compound error than capability. The dangerous

Table 7: ■ **SPECULATION** The risk atlas: what, and the defence that answers it. Speculative, deliberately non-operational.

Risk	Defender's toolbox
Weight-level backdoors	provenance + signed diffs, diff review, weight-space anomaly scans, behavioural red-teaming
Safety-training removal	tamper-resistant training, release governance, monitoring derivative checkpoints
Compute-threshold blind spot	capability-based evaluation, post-release change tracking
Recursive self-improvement	human-in-the-loop sign-off, sandboxed editors, immutable audit logs, self-edit rate limits
Licence laundering	weight attestation, watermarking research, model-card diffs
Hidden-capability concealment	multiple independent readers, ground-truth probe sets, reader calibration
Benchmark gaming	held-out, author-blind tasks (WSP-Bench-Lite's core rule)
Over-trust in explanations	verify every narrative against a measurement
Power concentration	open replications on small models, public benchmarks
What could go right	cheaper customisation, accessible interpretability, safer unlearning — fund these too

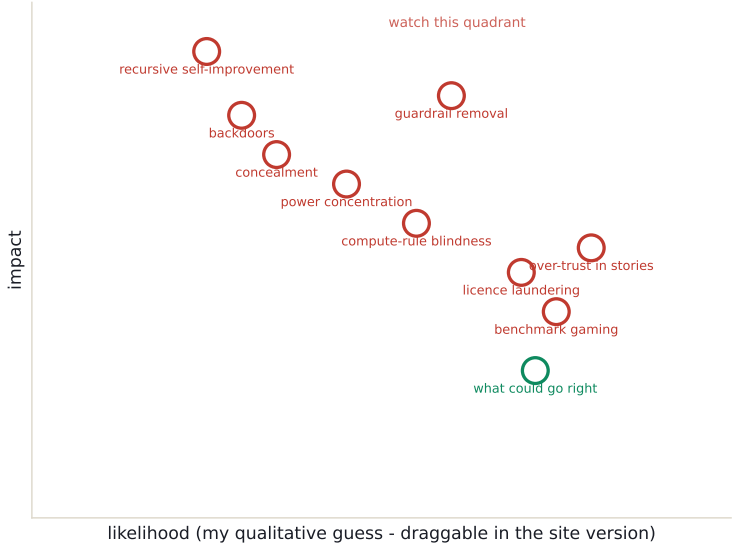


Figure 12: ■ **SPECULATION** The risk atlas as a likelihood×impact map (my qualitative placement; the site version is draggable so readers can disagree).

scenario needs a trusted automated evaluator first, which is a monitoring problem, and monitorable. Human sign-off and rate-limited self-edits remain cheap, effective controls while the science catches up.

12 Het's Lab: Three Runnables and WSP-Bench-Lite

■ **TEACHING SCHEMATIC** This section is my own contribution, clearly marked. Three labs (all under fifteen lines; verified running on Python 3.12 with NumPy 2.x; each finishes in under a second) make the mechanisms tangible. Lab 1 is the hand-compiled paddle follower of Eq. (11); Lab 2 performs the direction surgery of Eq. (8) and prints both the exact residual ($\max|u^\top W'| \approx 10^{-16}$) and the sparsity/energy trade-off when only the top-k columns are edited (remaining u-energy ≈0.74 at k=8 of 64); Lab 3 computes the flagship fraction of Eq. (9) (0.0164%, which the post rounds to 0.016%) and the $2 \times 64 \times 1024$ arithmetic behind 131,072, with its [VERIFY] flag preserved. The full sources ship in the package (`lab/` folder) and in Appendix D.

■ **SPECULATION** **WSP-Bench-Lite** is a community benchmark proposal — a proposal, with no results yet, because producing results is the community's job. Every rule answers a gap the independent review named (Table 8): three tracks (Read/Fix/Write); tasks authored by non-competing teams and held private between evaluation windows; ≥5 seeds per condition; multiple LLM participants; mandatory baselines (decoding tweaks, standard direction ablation, human interpreter, and the same heuristic as ordinary code); metrics that include *edit sparsity* s (Eq. 9), *functional drift* D (Eq. 12), the budget-curve fidelity of Eq. (10), and cost; pre-registered prompts [43]; public logs; and rotating task stewardship in the pattern of capture-the-flag competitions [44]. Reasoning traces are logged for study but never counted as evidence — only the measured Δ and its effects score.

Table 8: ■ **SPECULATION** WSP-Bench-Lite: the one-page scoring template (proposal v0.1, Het Patel).

Track	Task type	Mandatory baselines	Metrics
READ	identify behaviour from weights	human interpreter; same heuristic as code	accuracy, calibration, FPR
FIX	repair a seeded defect	decoding tweak; direction ablation; random-direction control	success, sparsity s, drift D
WRITE	produce a working policy from a spec	random init + short training; best published checkpoint	score, fidelity(n), cost

Implementation notes for harness builders, so the protocol survives contact with reality: targets ship as seeded generators (script plus seed), never as frozen checkpoints, so no participant can overfit one training run; FIX defects are injected by known transformations (sign flips, dead columns, looped directions) so ground truth for "what changed" exists; the drift probes are a frozen, task-unrelated set shipped with each task so participants cannot game D; every attempt publishes prompt, trace, Δ coordinates, budget curve, and seeds as JSONL. The whole v0.1 — thirty tasks across three tracks at ≤270M scale — is a few GPU-weeks; the expensive part is human, recruiting non-competing task authors and holding task sets private between evaluation windows [44]. Governance: rotating stewards, tasks relicensed CC BY 4.0 after retirement from the live set.

The two results v0.1 should chase first, both informative either way: (1) does any participant beat the standard direction-ablation baseline on FIX with statistical separation? (If not, the

field's flagship demo is an interface to known surgery — a finding that reframes everything, and is worth having.) (2) Does WRITE performance survive author-blind, procedurally novel task families? (If not, recall carried the games; if yes, the generality claim graduates to evidence.)

13 Limitations of This Guide

This document inherits the source's limits and adds its own. ■ **INDEPENDENT REVIEW** Inherited: every experiment is n=1; baselines are absent; tasks were author-designed; the primary sources could not all be independently re-opened at build time (fact-check date 5 Nov 2026), so paper-reported figures are transcriptions of the post and its released logs, with every unresolved item listed in Appendix C. ■ **TEACHING SCHEMATIC** Added by me: the qualitative ratings in Tables 1 and 6, the risk placements of Figure 12, the thermometer encoding of Figure 9, and the entire horizon map are editorial devices — clearly badged — not measurements. Where my reading of a mechanism could differ from the post's, the text says so (the direction-removal form is explicitly flagged as a teaching approximation). The reader should treat this whitepaper as a carefully sourced secondary work: cite the original post for results, the reviews for verdicts, and this guide only for its teaching apparatus, labs, and benchmark proposal.

14 Ethics and Responsible Use

This guide deliberately contains no operational instructions for harmful edits: the risk atlas names mechanisms at conceptual level only, and every risk is paired with defences. The dual-use asymmetry is acknowledged plainly — offence in this space is surgical and quiet; defence is infrastructural (signing, scanning, publishing) and, historically, slower. That lag, not any single attack, is the real risk, which is why the defensive research agenda (edit-detection theory, capability-based evaluation standards, attestation formats, reader calibration) is listed in Section 11 as fundable, publishable work. On the beneficial side: cheaper customisation for people without data budgets, interpretability that reads like a mechanic's report, unlearning that removes rather than papers over. The honest posture is neither doom nor hype but *defence-first curiosity* — which is also the practical recommendation of this document.

15 Conclusion

The claim of weight-space programming is extraordinary in the literal sense: a general-purpose reasoner reading a parameter vector and writing a working fix, with explanations, at a blast radius of 0.016%. The evidence assembled here supports a precise, modest version of it: *for small-to-mid networks and diagnosed mechanisms, reasoned edits and even whole-network writes reach quality milestones orders of magnitude cheaper than gradient descent* — with single-run evidence, missing baselines, and honest negative controls. The authors chose the word "sparks"; the word is load-bearing. What would convert sparks into fire is exactly what Section 12 proposes: author-blind tasks, seeds, baselines, drift and sparsity metrics, public logs. Until then, the right stance — held consistently by this guide's badge system — is calibrated fascination: read the artefacts, run the labs, demand the measurements, and design the next experiment.

A Appendix A — The Fact Sheet

■ **PAPER-REPORTED** Structured source data, also shipped as `research/fact-sheet.json` in the package and served by the companion site at `/data/fact-sheet.json`. Source: the blog post of 2 Nov 2026 [1]; model used: GPT-6 Astra; all logs released [33]; edited model on Hugging

Face [34]; live demo [35]; benchmark and harness "currently being built"; the post is **not** peer-reviewed. Fact-check date of this guide: 5 Nov 2026.

#	Verb	Weights	Reported headline
1	Read	17,253	Named all 4 games from 10×10 grids; Breakout at 99% confidence; no forward passes
2	Read	23,824	All 3 shapes identified; described outlines while the models draw filled shapes
3	Read	7.16M	Language 6/6 (confidence 0.48–0.62); copying heads exactly match ground truth: Python 2.1/3.1/3.5, romaji 3.3, English 3.1
4	Fix	171,664	One-shot edit of 3.5% of weights: ramp success 0/256 → 1,024/1,024
5	Write	≈20,486/net[a]	MinAtar policies from zero: 517.4/53.1/53.2 vs PPO baselines (Table 4)
6	Write	1,480,274	9×9 Go from zero: 54–55% at 0 passes vs small PPO; 70–79% at 50 passes vs equal-size PPO (Figure 8)
7	Read	∼270M×2	Weights alone 0/4; with attention patterns 3/3; head 10.2: 94.3/0.75 vs 3.5/85.7
8	Fix	131,072	Direction subtracted from MLP outputs of layers 15 and 19, 64 columns each: MATH-500 7.9 → 48.1%, token-limit hits 93 → 33.1% at 50 passes (Table 3); Gemma control flat

[a] Computed by this guide from the stated architecture, not reported by the authors.

B Appendix B — Glossary (Abridged)

A 60-term glossary ships with the companion site; the twenty-four most load-bearing terms, one line each. θ: the vector of all parameters. **Weight space**: the space of all possible θ. **Parameter count** P: how many numbers a model contains. **Gradient descent**: today's default writer (Eq. 2). **Learning rate** η: the step size of training. **Forward pass**: one run of the network on an input. **Receptive field**: the input slice a weight responds to — why a 10×10 grid can show a paddle row. **Permutation symmetry**: hidden units can be relabelled without changing the function — why naive weight-reading is underdetermined. **Hypernetwork**: a network trained to output weights [12]. **Fast-weight programmer**: a transformer that writes temporary weights as it runs [13]. **Tracr**: a compiler from formal programs to weights [15]. **WSP**: reading, fixing and writing θ by reasoning (Eq. 3). Δ: the edit. **Spec**: the task description conditioning the edit. **Feedback budget** n: forward passes the editor may run. **Task vector** τ: fine-tuned minus base (Eq. 5). **Induction head**: the two-head copying circuit behind in-context learning (Eq. 7). **Copying head**: the heads Exp. 3 identified from weights alone. **Direction removal**: the projection of Eq. 8. **Edit sparsity** s: the blast radius (Eq. 9). **Functional drift** D: the containment measure (Eq. 12). **Token-limit share**: the fraction of answers that ram the generation ceiling — the mechanism check behind the flagship fix. **Negative control**: a run designed to show no effect; Gemma's flatness is what validates the flagship claim. **Author-blind task**: a benchmark task created by someone other than the team being evaluated — the core rule of WSP-Bench-Lite.

C Appendix C — Open Verification Items

The honesty ledger: everything this guide could not independently confirm at build time (fact-check date 5 Nov 2026).

1. The primary post URL and the two review URLs could not be re-opened at build time; all paper-reported figures are transcriptions from the source fact sheet and the released logs.
2. The hidden size of 1024 behind 131,072 = 2 × 64× width is inferred by arithmetic, flagged [VERIFY] where used.
3. The weight-space-learning survey identifier (arXiv:2603.10090) is as given in the source post; not independently openable.
4. The Gemma 3 / Qwen3 technical-report identifiers and the Kinetix author list are pending confirmation.
5. The Muon optimizer has an informal citation; to be pinned before this bibliography is cited as final.
6. Three lineage entries (steering, sparse-pruning, delta-debugging) were *dropped* from this document's bibliography pending exact citations; restore them with pinned citations in v1.1.
7. All experiment tables have a single source (the post); the released logs are the cross-check and are cited for that purpose.
8. The MinAtar per-net count 20,486 is *computed* by this guide from the stated architecture, not reported by the authors.

D Appendix D — The Three Labs

■ **TEACHING SCHEMATIC** Verified runnable (Python 3.8+, NumPy only). Shipped in `lab/` with a README.

Listing 1: lab/01_compile_a_rule.py — hand-compile a paddle follower (teaching schematic).

```
import numpy as np
relu = lambda z: np.maximum(z, 0.0)
W1 = np.array([[-1., 1.], [1., -1.]]); b1 = np.array([-0.05, -0.05])   # h0: ball
    left of paddle, h1: ball right
W2 = np.array([[1., 0.], [0., 0.], [0., 1.]]); b2 = np.array([0., 0.05, 0.])
ACTIONS = ["left", "stay", "right"]
def act(ball, paddle):
    h = relu(W1 @ np.array([ball, paddle]) + b1)
    return ACTIONS[int(np.argmax(W2 @ h + b2))]
assert act(0.2, 0.7) == "left" and act(0.8, 0.3) == "right" and act(0.5, 0.52) == "
    stay"
```

Listing 2: lab/02_direction_surgery.py — remove one direction from an MLP output matrix.

```
import numpy as np
rng = np.random.default_rng(0)
d_model, d_ff = 16, 64
W = rng.normal(size=(d_model, d_ff))
u = rng.normal(size=d_model); u /= np.linalg.norm(u)
W_full = W - np.outer(u, u @ W)                          # (I - u u^T) W
print("max |u^T W'| =", np.abs(u @ W_full).max())       # ~ 1e-16
k = 8; cols = np.argsort(-np.abs(u @ W))[:k]             # sparse variant: edit only
     k columns
W_sp = W.copy(); W_sp[:, cols] -= np.outer(u, u @ W[:, cols])
print(f"edited {k}/{d_ff} columns; remaining u-energy "
      f"{np.linalg.norm(u @ W_sp) / np.linalg.norm(u @ W):.2f}")  # ~0.74
```

Listing 3: lab/03_blast_radius.py — the sparsity arithmetic of the flagship edit.

```
edited, total = 131_072, 0.8e9
```

```
print(f"{edited/total:.4%}")                                    # 0.0164%  (paper rounds to
    0.016%)
print(2*64*1024)  # 131072 = 2 layers x 64 columns x 1024 (hidden size = inference,
     VERIFY)
```

E Appendix E — Task Schema and Reproducibility Checklist

■ **SPECULATION** JSON task schema (v0.1), one file per task; the sha256 of the prompt is pre-registered before any run:

Listing 4: Task schema, condensed.

```
{ "task_id": "FIX-kine-ramp-01",
  "track": "FIX",
  "author_team": "alpha",        // NEVER the evaluated team
  "target": { "family": "mlp", "shape": [679,128,640,16],
              "init": "ppo_trained", "defect": "seeded: ramp_thruster_sign_flip" },
  "spec": "The car fails the ramp. Fix it.",
  "budget": { "forward_passes": 10, "reasoning_tokens": null },
  "baselines_required": ["random_dir_ablation", "standard_dir_ablation",
                         "decoding_tweak", "human_interpreter"],
  "metrics": ["success_rate", "edit_sparsity_s", "functional_drift_D",
              "budget_curve_fidelity_n", "cost_usd"],
  "seeds": 5,
  "report": { "pre_registered_prompt": "sha256:...", "public_logs": true } }
```

Reproducibility checklist for any WSP claim, distilled from this guide's quality gates:

1. Does every number bind to a published table or log?
2. Does every figure carry exactly one evidence badge, and do schematics say so?
3. Are all baselines run — decoding, standard direction ablation, plain code, human?
4. Are there $\geq$5 seeds, with spread reported and no smooth interpolation between budget points?
5. Is there a negative control, and a mechanism check that ties the narrative to a measurement?
6. Is the edit sparsity s reported, and the functional drift D bounded on held-out behaviour?
7. Were the tasks authored by someone other than the evaluated team?
8. Are the prompt, the reasoning trace, and the Δ coordinates published?

References

[1] Rikiya Takehi, Jayden Teoh, Alex Zhang, and Omar Khattab. Sparks of weight-space programming: LLMs can natively read & write neural network weights. Blog post, MIT CSAIL (rikiyat.github.io), 2026. 2 Nov 2026. The primary source for every paper-reported figure in this guide.

[2] Andreas Maier. Weight-space programming: what is measured and what is anecdote? Substack (akmaier.substack.com), 2026. 5 Nov 2026. The sorted-evidence scoreboard behind this guide's independent-review badge.

[3] Sean Goedecke. LLMs are not superhuman at reading and writing neural network weights. Essay, seangoedecke.com, 2026. The deflationary case across the post's experiments.

[4] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.

[5] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.

[6] Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. In *International Conference on Learning Representations*, 2019. arXiv:1803.03635.

[7] Samuel K. Ainsworth, Jonathan Hayase, and Siddharth Srinivasa. Git re-basin: Merging models modulo permutation symmetries. In *International Conference on Learning Representations*, 2023. arXiv:2209.04836.

[8] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.

[9] Keller Jordan, Jeremy Bernstein, et al. Muon: An optimizer for hidden layers in neural networks. Community optimizer (blog/GitHub), 2024. Informal citation pending pinning (Appendix C).

[10] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, 2020.

[11] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. arXiv:1707.06347, 2017.

[12] David Ha, Andrew Dai, and Quoc V. Le. HyperNetworks. In *International Conference on Learning Representations*, 2017. arXiv:1609.09106.

[13] Imanol Schlag, Ilya Tolstikhin, Roland Buesing, Valerii Likhosherstov, Anirudh Goyal, Yee Whye Teh, et al. Linear transformers are secretly fast weight programmers. In *International Conference on Machine Learning*, 2021. arXiv:2102.11174.

[14] Ziwei Wang, Xindi Wang, Yushun Tang, Qinying Guo, Zhiqi Shen, Le Feng, Lei Feng, Xinwang Liu, En Yu, Kai Ni, Peng Wang, Yuexuan Tu, Bei Hua, Xiu Li, Yong Luo, Lu Wan, Jun Wang, Xinglong Chang, Kejun Wang, and Xiaochen Zhang. Neural network diffusion. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024. arXiv:2401.04012.

[15] Martin Lindell, David Ramage, Christian Herold, Alex Barbero, Daniel Balle, James Chua, et al. Tracr: Compiled transformers as a laboratory for interpretability. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2301.05062.

[16] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. arXiv:2106.09685.

[17] Roei Ilharco, Mitchell Wortsman, Samir Yitzhak Gadre, Brian Chen, Rebecca Roelofs, et al. Editing models with task arithmetic. In *International Conference on Learning Representations*, 2023. arXiv:2212.04089.

[18] Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. TIES-Merging: Resolving interference when merging models. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2306.01708.

[19] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems*, 2022. arXiv:2202.05262.

[20] Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, et al. A mathematical framework for transformer circuits. Transformer Circuits Thread, 2021.

[21] Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Mingchen Dong, Tom Henighan, et al. In-context learning and induction heads. Transformer Circuits Thread, 2022.

[22] Neel Nanda, Lawrence Chan, Tom Lieberum, George Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability. In *International Conference on Learning Representations*, 2023. arXiv:2301.05217.

[23] Authors as cited in the source post. Weight space learning: a survey. arXiv:2603.10090, 2026. Identifier as given in the source post; not independently openable (Appendix C).

[24] Community. Awesome-weight-space-learning. Curated GitHub repository, as named in the source post, 2026.

[25] Andy Arditi, Oscar Obeso, Syed Ather Shah, Nauman Ahmad, Maryam Jahanzeb, Samyak Dhar Jain, et al. Refusal in LLMs is mediated by a single direction. In *Advances in Neural Information Processing Systems*, 2024. arXiv:2406.11717.

[26] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, 2020.

[27] Gemma Team, Google DeepMind. Gemma 3 technical report. arXiv preprint (Gemma series), 2025. arXiv identifier pending confirmation (Appendix C).

[28] John Arnold, Ben Ellis, et al. Kinetix: Investigating the training of agents through open-ended physics-based control tasks. arXiv preprint, 2024. Author list pending confirmation (Appendix C).

[29] Qwen Team. Qwen technical report. arXiv preprint (Qwen series), 2025. Benchmark variant names as in the source post; identifier pending confirmation (Appendix C).

[30] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *International Conference on Learning Representations*, 2022.

[31] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, et al. Training verifiers to solve math word problems. *arXiv:2110.14168*, 2021.

[32] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Suvinay Subramanian, Sean Ladh, Kevin Hui, et al. Instruction-following evaluation for large language models. In *Annual Meeting of the Association for Computational Linguistics*, 2024. arXiv:2311.07911.

[33] Rikiya Takehi et al. lm_understand-logs. GitHub repository, 2026. Released experiment logs, github.com/RikiyaT/lm_understand-logs.

[34] Rikiya Takehi et al. Qwen3.5-0.8b-weight-space-programming. Hugging Face model repository, 2026. The edited checkpoint released by the post's authors.

[35] Ahsen Khaliq. weight-space-programming-demo. Hugging Face Space (live demo), 2026.

[36] Kenny J. Young and Tian Tian. MinAtar: An Atari-inspired testbed for thorough and reproducible reinforcement learning experiments. *arXiv:1903.03176*, 2019.

[37] Simon Willison. Qwen can interpret model weights. Blog post, simonwillison.net, 2026. 3 Nov 2026. Independent laptop-class replication of a similar weight-reading example; anecdotal.

[38] Collin Burns, Haotian Ye, Dan Klein, and Jacob Steinhardt. Discovering latent knowledge in language models without supervision. In *International Conference on Learning Representations*, 2023.

[39] Samuel Marks and Max Tegmark. The geometry of truth: Emergent linear structure in LLM representations of true/false datasets. *arXiv:2310.06824*, 2023.

[40] Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *ACM Conference on Fairness, Accountability, and Transparency*, 2019.

[41] SPDX and CycloneDX communities. Software bill of materials (SBOM) standards. Linux Foundation / OWASP standards, 2021. The template for a hypothetical "weights BOM".

[42] Nicholas Carlini, Chang Liu, Úlfar Erlingsson, Jernej Kos, and David Song. Extracting training data from large language models. In *USENIX Security Symposium*, 2021.

[43] Center for Open Science. Preregistration standards (OSF). osf.io/registries, 2019. The model for WSP-Bench-Lite's pre-registered prompts.

[44] CTF communities (DEF CON, picoCTF). Capture-the-flag competition governance: rotating task stewardship. Community practice, 2023. Governance footnote for benchmark stewardship.